

๑. 2 เพราะถ้า I/O แบบซีโรนีส โปรแกรมเมอร์ จะพร้อมสำหรับข้อมูลในแง่จาก อุปกรณ์ I/O เลขในในพอร์ทที่อุปกรณ์ ส่งข้อมูลมา จะรู้ว่ามันเป็นตัว มีบิตบว

๑. 4 A ซีโรนีส

B ๐ ซีโรนีส

C ซีโรนีส

ถ้าผู้รับ การติดตาม รู้ได้ทันที คือ ซีโรนีส แต่การ การตามของ ขึ้นอยู่กับสถานะที่ผู้รับคนแรก ไม่รู้ เหมือน คือ ๐ ซีโรนีส

๑. 5 บิตแต่ละบิตของ ระบุว่าค่าใน KBR บิต ถูกหรือไม่

1 = พร้อม KBR มีรหัส ASCII ของผู้ใช้ที่กดล่าสุด

0 = ว่างพร้อม ค่าใน KBR ว่างถูกต้อง

๑. 10 อาจจะเป็นคนที่บอกคนที่ส่งไปก่อนหน้านั้น เสร็จแล้ว เสร็จในผู้ใช้เนี่ย

๑. 11 โดยทั่วไปแล้ว I/O ที่จับคู่กันดีจขงจั้น เพื่อรับส่ง ประสิทธิภาพ มากกว่า เปรียบเทียบ จาก CPU สามารถทำงานอื่นๆ ได้ ในพอร์ท หรือ I/O แทนที่จะ (ช้ากว่า) ในการ โยนคำสั่งจาก CPU เข้า การโอนข้อมูล ประสิทธิภาพ ในการโอนนั้น เพราะกับอุปกรณ์ ที่เร็วมาก ซึ่งตัวอุปกรณ์ทางสอง เทคโนโลยีนี้นับว่าดี

๑. 12 a มีแผ่นจากภายนอกมาทำหน้าที่อ่านหรือเขียนข้อมูลจากที่เครื่องหยุดทำงาน (HALT)

b ตัวค่า 0 เป็น MCR[5] เพื่อขยับการคำนวณของเครื่อง

c 0: เริ่มจาก บวกที่ 13 ซึ่ง เป็น บวกที่เกิดเหตุการณ์ HALT จี๊วๆ

d จะส่งกลับค่าที่มันส่งค่าแล้ว LALT ที่ทำงาน เสร็จดำเนินการส่งกลับค่าแล้วกลับไป

๑. 18 a 1111 0000 0010 0001

b 0011 0010 0000 0011

c 1100 0001 1100 0000

d Hookem Horns

๑. 21 เป็นโปรแกรมตรวจสอบว่า A เป็นจำนวนเฉพาะหรือไม่ ถ้า RESULT เป็น 1 ก็คือ 0 ไม่เป็น

๑. 37 Interrupt ของ D เกิดจากค่าแล้ว x 6310 เสร็จ

CPU เก็บ PC ของ ISR C ลง stack กระโดดไป ISR D (x1400) แล้วจบ RTI ก็จบการทำงาน ISR
เมื่อ ISR C จบ → RTI → กลับไปทำงานตามลำดับปกติ

c ต่อ

14.1 โจรกรรมทั้งหมดในภาษา C จะต้องเริ่มจากฟังก์ชัน main

14.2

- A เก็บจุดเริ่มต้นของตัวแปร local ของฟังก์ชัน
- B เก็บค่าที่ส่งกลับ (return) ในตำแหน่งที่ฟังก์ชัน ที่ถูกเรียก CPU จะรู้ ตำแหน่งที่จะทำงานต่อหลังจากฟังก์ชันจบ
- C ค่าที่ส่งกลับจะส่งกลับของฟังก์ชันไปยังฟังก์ชันที่เรียก

14.6 1 2 6 3 4

14.9 อาริ กิวเมนต์แรก จะถูกวางไว้บนสแต็ก จากนั้นตัวเรียกคือ JSR
เนื่องจากพารามิเตอร์ ใน math ตัวแรกอยู่ นอกขอบเขตของฟังก์ชัน
ตัวเรียกจึงต้องส่งพารามิเตอร์ ก่อนว่า การควบคุมจะไปถูกผู้เรียก